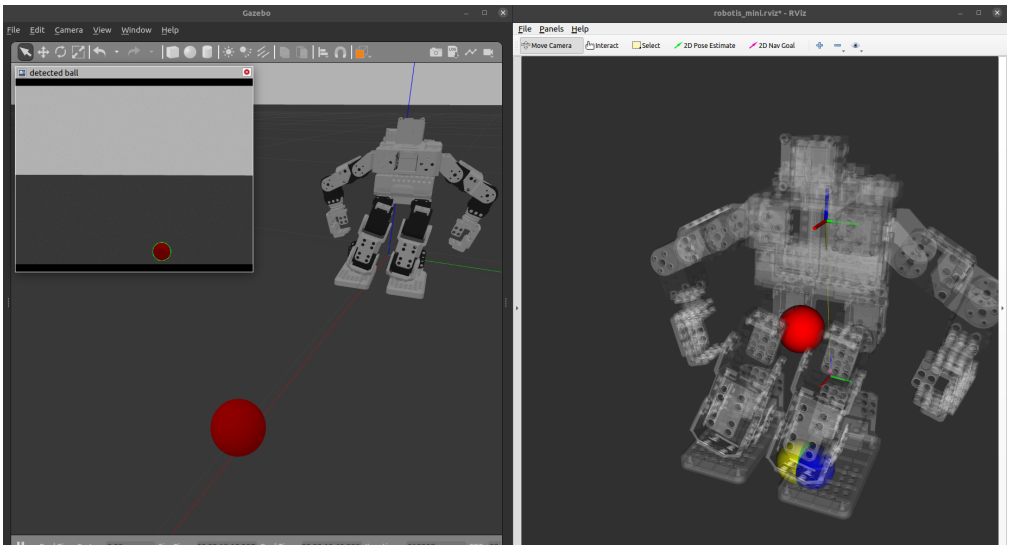
ECE 598JK

Assignment 4 Instruction



Assignment 4 is an individual task. Please submit the following in a single zip file:

- **three videos** of simulation results demonstrating walking motion 1) without IMU feedback 2) with IMU feedback and 3) with IMU + Camera feedback. The video should **include measured ZMP and approximated ZMP visualizations** for the walking motion in rviz.
- **a video** of hardware result demonstrating walking motion with IMU + Camera feedback. This video does not need ZMP visualization.
- **five files** you edited for the assignment
  - approx_zmp_multi_mass.py
  - approx_zmp_single_mass.py
  - measured_zmp.py
  - walkingParam.yaml
  - ball_detection.py
- a PDF document including answers and discussions for the given questions

Follow the instructions from the link below to setup your ROS workspace and assignment starter code in both simulation and hardware. If done correctly, you should see the Robotis MINI model standing in front of a red ball in the Gazebo simulation and Rviz visualization.

`https://github.com/uiuckimlab/mini-ros-assn4`

Q1 Measured ZMP (5 Points)

ZMP can be measured using the 6 axis force/torque sensors located at the foot of the robot. The sensors are coordinated to simultaneously measure the force $f = [f_x, f_y, f_z]$ and moment $\tau = [\tau_x, \tau_y, \tau_z]$ applied from the ground to the sole.

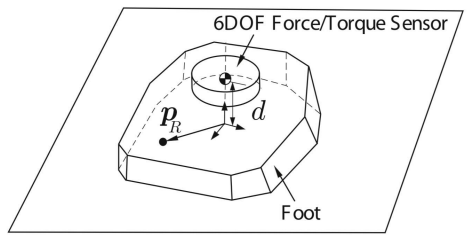


Figure 1: Definition of Variables for Calculation of ZMP by 6 Axis Force/Torque Sensor

From the measured sensor data, the ZMP of a single foot can be calculated by

$$\begin{aligned} p_{Rx} &= \frac{-\tau_y - f_x d}{f_z} \\ p_{Ry} &= \frac{\tau_x - f_y d}{f_z} \end{aligned} \tag{1}$$

When both feet are making contact with the floor, the ZMP position can be calculated using each foot's measured ZMP as the following equation.

$$\begin{aligned} p_{x,both} &= \frac{p_{Rx} f_{Rz} + p_{Lx} f_{Lz}}{f_{Rz} + f_{Lz}} \\ p_{y,both} &= \frac{p_{Ry} f_{Rz} + p_{Ly} f_{Lz}}{f_{Rz} + f_{Lz}}. \end{aligned} \tag{2}$$

Please refer to Chapter 3.2 of the textbook [1] for more information about measured ZMP.

The first task is implementing measured ZMP calculation for the Gazebo simulation model and visualizing it in Rviz. In the provided python starter code, force sensors are already implemented and low-pass filters are applied to the sensor output signals to filter out high frequency noise. Using the sensor readings and Eqn. (1), (2), implement the ZMP calculation and visualization (use a blue colored, 1.5cm radius spherical solid ball) in Rviz.

Q2 Approximated ZMP (5 Points)

ZMP can be approximated using a simplified model and its dynamics. Figure 2 presents two simplified models used to approximate ZMP. Please refer Chapter 3.4 in [1] for more information.

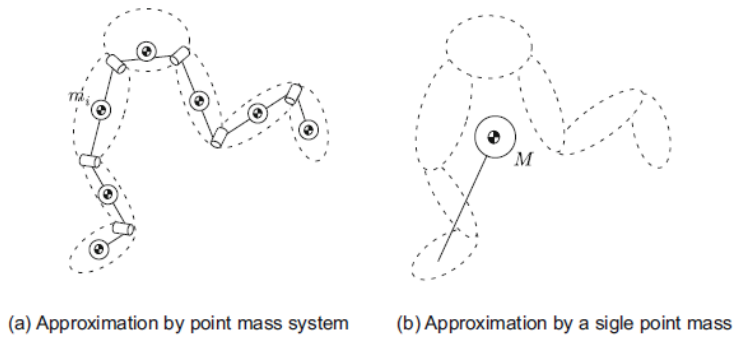


Figure 2: Approximated Multibody system [1]

1) ZMP approximation by multi-mass model

As shown in Fig. 2 (a), a multi-mass model assumes each link of the robot as a point mass located at the CoM of the link. The effect of the inertia tensor of each link are neglected in the model, so it simplifies the calculation for angular momentum. With the multi-mass model, the ZMP can be expressed as

$$p_x = \frac{\sum_{i=1}^{N} m_i\left((\ddot{z}_i + g)x_i - (z_i - p_z)\ddot{x}_i\right)}{\sum_{i=1}^{N} m_i(\ddot{z}_i + g)} \qquad p_y = \frac{\sum_{i=1}^{N} m_i\left((\ddot{z}_i + g)y_i - (z_i - p_z)\ddot{y}_i\right)}{\sum_{i=1}^{N} m_i(\ddot{z}_i + g)} \tag{3}$$

Here, m_i and $[x_i, y_i, z_i]$ are the mass and the position of i_{th} CoM, and g is gravitational constant.

Using the provided starter code in approx_zmp_multi_mass.py, calculate approximated ZMP by the multi-mass model and visualize it (use a yellow colored, 1.5cm radius spherical solid) in Rviz.

2) ZMP approximation by single-mass model

In the single-mass model approximation, the whole body of a robot is assumed as a point mass at the whole body CoM is located as shown in Fig. 2 (b). In this model, ZMP can be calculated from the position and acceleration of the following equations:

$$\begin{aligned} p_x &= x - \frac{(z_{CoM} - p_z)\ddot{x}_{CoM}}{\ddot{z}_{CoM} + g} \\ p_y &= y - \frac{(z_{CoM} - p_z)\ddot{y}_{CoM}}{\ddot{z}_{CoM} + g} \end{aligned} \tag{4}$$

Using the provided starter code in approx_zmp_single_mass.py, calculate approximated ZMP

by the multi-mass model and visualize it (use a red colored, 1.5cm radius spherical solid) in Rviz.

Q3 ZMP Walking with Feedback (10 Points)

1) Walking motion without IMU feedback

In this task, you need to find parameters to make the Upgrade MINI walking forward without IMU feedback. The instructions to execute the walking motion on the robot in simulation and hardware can be found on the github readme.

You can adjust the parameters for the walking motion in the configuration file. The configuration file is located in `/ws_mini_assn4/src/mini-ros-assn4/mini_ros/config` and the file name is "`walkingParam.yaml`".

In **simulation**, the given parameters are `x size = 20, y size = 30, n step = 6, z height = 8.0`. On the **hardware**, the given parameters are `x size = 10, y size = 35, n step = 6, z height = 25.0`. Adjust the parameters for **step time** ($0.4 \leq$ `t_step` ≤ 1.5) and **double support phase ratio** ($0 \leq$ `ds_time` ≤ 0.9) so that the robot make 6 steps without falling down.

Due to the deformation of the mechanical parts of the robot, the robot's walking motion may not be perfect. However, make sure the robot completely lift its swing foot off the ground, and try to avoid too much slipping on the support foot.

Also, make sure you use **FULLY CHARGED** batteries when you are taking a submission video. The external power cord will be useful when you spend a long time on the robot, but it affects to the robot's motion as well.

You don't need to compile any code, just change the parameters in the YAML file and save it. The changed parameters will be automatically applied when you run the launch file.

2) Walking motion with IMU feedback

Now, let's enable the stabilizer while running the walking motion. The simple stabilizer algorithm is configured as shown in Figure 3. The stabilizer feeds back the orientation (roll, pitch) to the foot positions and the ankle angles with proportional gains to compensate for the orientation deviation. In addition, the stabilizer feeds back y accelerations to the foot y positions.

In this task, you need to tune the stabilizer gains to make the robot walking stably by adjusting the following parameters in the YAML file: $-0.1 <$ `pitch_ankle_gain` < 0, $-0.1 <$ `roll_ankle_gain` < 0, $0 <$ `roll_disp_gain` < 50, $0 <$ `ay_gain` < 5.0. Set the other parameters to the values referenced above.

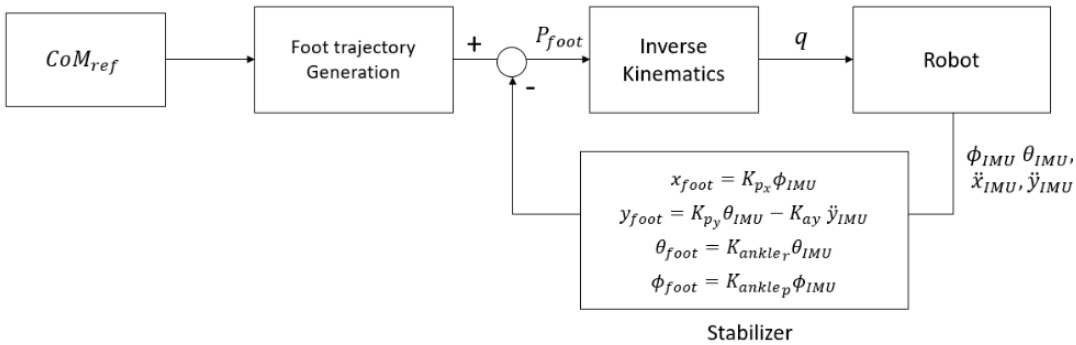


Figure 3: Stabilizer Algorithm using IMU orientation and linear acceleration

3) Walking motion with Camera + IMU feedback

Now, let's enable the perception feedback used for walking pattern generation by setting `use_perception` to `True` in "`walkingParam.yaml`". In this task, you need to adjust the blob detector parameters in `ball_detector.py` to detect the target object. The estimated distance of the detected target object is used to make the robot stop in front of the object by dynamically setting the number of walking steps needed to reach the target object in `zmp_walking.cpp`.

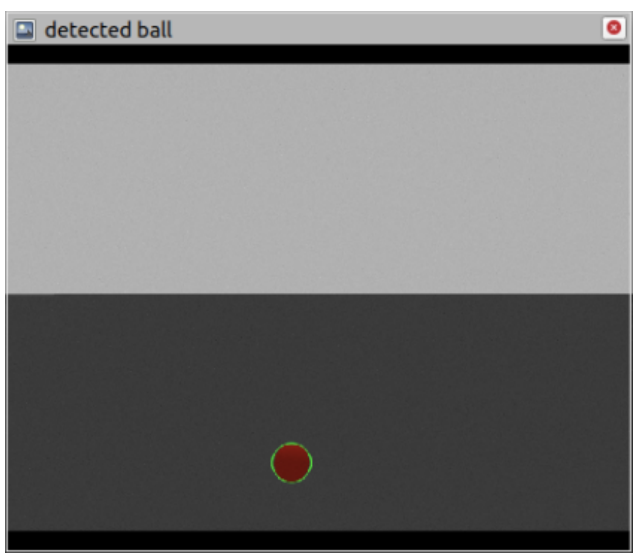


Figure 4: Perception output of detected target object

```
ROS_MASTER_URI=http://localhost:11311

process[ball_detection-1]: started with pid [161204]
process[walking-2]: started with pid [161205]
Calculating number walking steps from detected ball position...
[ INFO] [1669064873.664257592, 12.574000000]: Calculated num_step: 26
Calculating ZMP and CoM trajectories...
Press any key to start

Press any key to continue! (or press ESC to quit!)
[ INFO] [1669064903.261759694, 41.890000000]: Loop: 0.00 ms  Elapsed Time: 0.00 s
| ax: 0.00  ay: 0.00  az: 0.00   | pitch: 0.00    roll: 0.00
[ INFO] [1669064903.263183895, 41.892000000]: Loop: 1.45 ms  Elapsed Time: 0.00 s
| ax: 0.00  ay: 0.00  az: 0.00   | pitch: 0.00    roll: 0.00
[ INFO] [1669064903.266483089, 41.895000000]: Loop: 3.31 ms  Elapsed Time: 0.00 s
| ax: -0.05  ay: 0.00  az: 3.27   | pitch: 0.04    roll: -0.03
```

Figure 5: Terminal output for calc num steps from detected ball position

Appendices

A Walking Pattern Generation Script

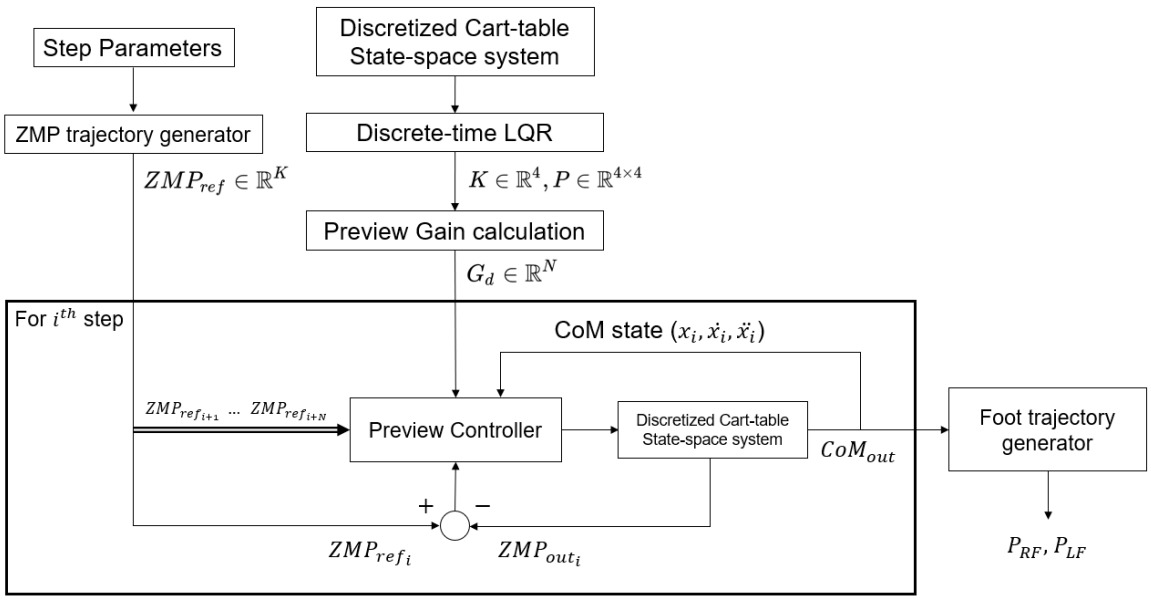


Figure 6: Diagram of the ZMP-based Walking Pattern Generator with a preview controller

Figure 6 shows the overall structure of the walking pattern generator.

According to a single mass, a constant height LIP model, the ZMP is given as

$$\ddot{x} = \frac{g}{z_c}(x - p) \tag{5}$$

where p is $[x_{zmp}, y_{zmp}]^T$, x is the position of the whole body CoM $[x_{CoM}, y_{CoM}]^T$ and z_c is a constant target height of the whole body CoM. In the cart-table model, we can rewrite this equation as follows:

$$p = x - \frac{z_c}{g}\ddot{x} \tag{6}$$

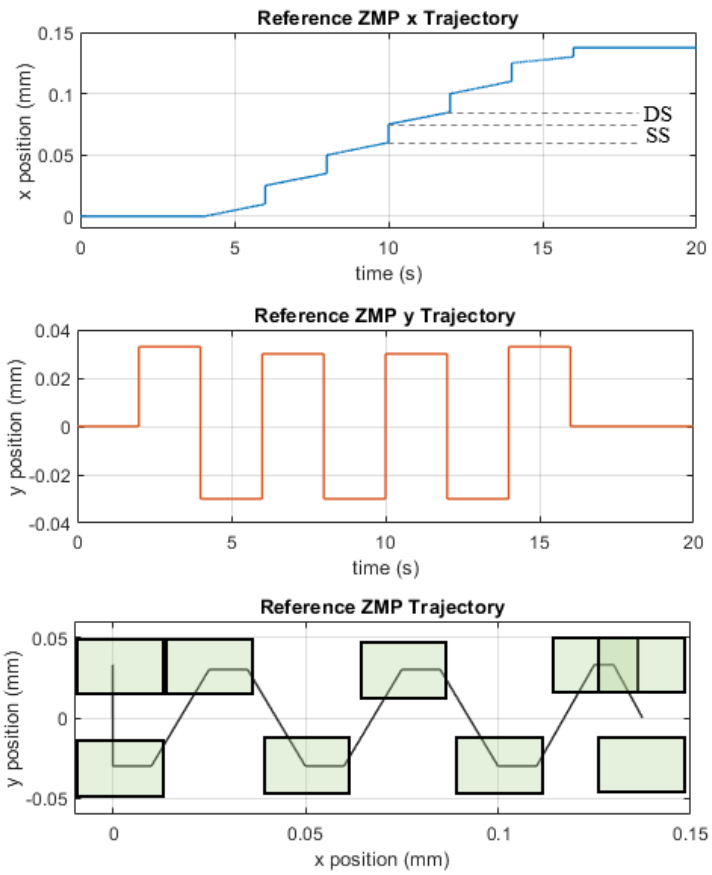


Figure 7: Reference ZMP trajectory

Figure 7 shows the ZMP trajectory generated by the defined variables. x trajectory is composed of step function for the single support phase and ramp function for double support phase. Then y trajectory is planned with step function to represent the lateral step location of each footstep. Note that we want peaks of ZMP trajectory located at the center of the supporting foot since that ensures the most safety margin.

By taking a derivative of the CoM acceleration ($\dddot{x}$) as a system input, and ZMP position as a system output, we can write a state space equation of ZMP and CoM as follows:

$$\frac{d}{dt}\begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} u \tag{7}$$

$$y = \begin{bmatrix} 1 & 0 & -\frac{z_c}{g} \end{bmatrix}\begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix} \tag{8}$$

Since we are simulating a discrete-time system, the system matrices need to be converted to

discrete-time system matrices for the sample time of Δt.

Now, using the discrete-time system matrices, apply a preview controller to track the reference ZMP trajectory by controlling the CoM of the LIP model. The preview controller is one of the optimal control strategies that use future reference to determine the optimal input to control the system [2].

There is a function script implementing the preview controller. The function takes the reference ZMP trajectory, the sample time and total simulation time, the discrete-time system matrices, the preview time, and finally Q and R matrix for the cost function of a Linear-Quadratic Regulator. The preview controller is implemented based on the method proposed in Chapter 4.4.3, Section 2 of the textbook.

The objective of the controller is to minimize the ZMP tracking error with the defined system by finding an optimal system input u_i at each time step.

$$u_i = -Kx_i + \begin{bmatrix} f_1 & f_2 & \cdots & f_i \end{bmatrix} \begin{bmatrix} y_{i+1}^{ref} & \cdots & y_{i+N}^{ref} \end{bmatrix}^T \tag{9}$$

We can observe that the system input consists of the state feedback and the feedforward of the future reference for the N steps of the preview time. A Linear Quadratic Regulator is used to find an optimal gain matrix K and the solution of a Ricatti Equation P for the discrete-system. The cost function for the LQR is defined as follows:

$$J = \sum_{n=1}^{\infty} Q(y_n^{ref} - y_n)^2 + Ru_n^2 \tag{10}$$

where Q is a penalty for tracking error and R is a penalty for the amount of input work. The LQR gives the optimal K and P which minimize the cost function J. Once we have K and P matrices, we can calculate a future reference weight f_i. Please refer to the textbook for the detailed equations.

Once we calculate the feedback gain and feedforward gain, we can simulate the system using the discrete-system matrices to get the CoM and resulting ZMP of each time step.

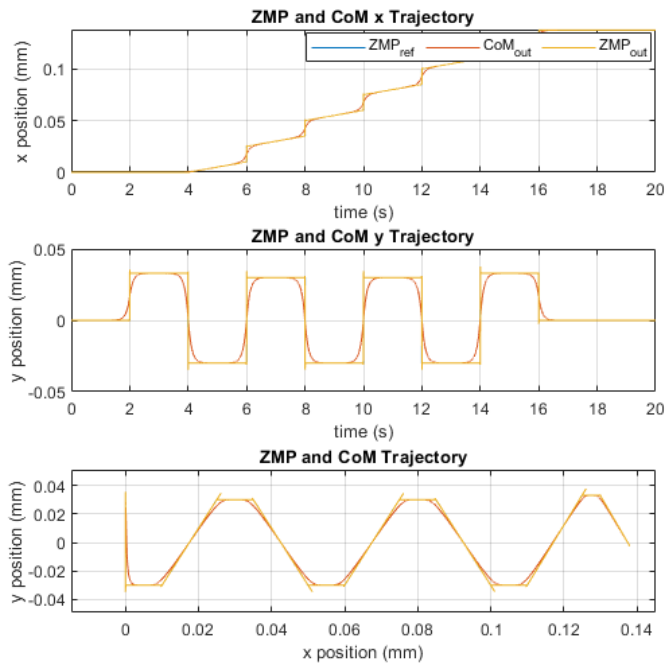


Figure 8: Result CoM and ZMP Trajectory

The CoM and ZMP results for the reference trajectory are shown in Figure 8. Now, a foot trajectory generator plans the trajectories of both feet to track the reference CoM and the planned step locations.

References

[1] Kajita, Shuuji, et al. Introduction to humanoid robotics. Vol. 101. Springer Berlin Heidelberg, 2014.

[2] Katayama, Tohru et al. Design of an optimal controller for a discrete-time system subject to previewable demand. International Journal of Control 41. 677-699, 1985.